Guidebook

Adapter service implements the following functionalities:

- Aggregates one or many Game Providers
- Connects with Remote Wallets via standard or custom API
- Multi currency including crypto
- Advanced permission system
- Fast, flexible and powerful reporting
- All operations available via GraphQL API
- API stitching to combine multiple endpoints into unified API with single authorisation
- Automated Integration Testing Tool

Structure & Naming

We use 3 level structure to describe where a player is coming from. In some cases when 2 level (or even 1 level) is enough `brand` can be null or same value as `operator` can be passed.

- `wallet` - defines Remote Wallet integration endpoint where the system is integrated too
- `operator` - defines name of the operating entity or a group
- `brand` - defines casino website. Parameter is optional

To define where `game` belongs to we can use:

- `rgs` - defines server-to-server integration
- `provider` - defines name of the gaming provider

Transactions

Statuses

- `started` - transaction has been initialised
- `finished` - transaction successfully closed
- `failed` - transaction failed
- `cancel` - transaction marked to be canceled. Only withdrawals can be cancelled
- `cancelled` - transaction successfully cancelled. Only withdrawals can be cancelled
- `rejected` - transaction rejected due to validation failure

Closing transactions manually

In case of repeatable problem on wallet side some transactions may get never closed. For this case there is GraphQL API to close them manually.

- if withdrawal gets stack in `cancel` state you will have option "Mark as cancelled". In this case wallet will need to cancel the bet and give money back to the player manually
- if deposit gets stack in `failed` state you will have option "Mark as finished". In this case wallet will need to pay the win to the player manually
- if deposit gets stack in `rejected` state you will have option "Mark as failed". In this case we will repeat the transaction automatically during next cycle

It should be used only when operator explicitly confirms it should be closed but for some reason their system can't return correct response.

Players

Players are created upon successful authentication with the wallet. Further authentications can update player's data but **currency cannot be changed** (it will be omitted even if wallet responds with different currency).

Wallets

Wallets are remote endpoints to manage player's funds and session.

Adding, removing or editing wallets requires restarting application.

Standard Adapter

Seamless wallet adapter documented here.

adapter: `standard` config:

```
{
  "secretKey": "abc",
  "url": "http:/example.com"
}
```

`

- `secretKey` : secret key for wallet authentication
- `url` : wallet endpoint. `${operator} will be swapped with` operator` variable passed in the launch url
- `timeout` (optional): request timeout in seconds (default `15` seconds)
- `transactionUsePost` (optional): if `true` it will use `POST` for `/transaction` requests
- `cancelUsePost` (optional): if `true` it will use `POST` for `/cancel` requests
- `overwriteGame` (optional): if value is specified it will overwrite game params with hardcoded value for all games (used for some multiplayer games)
- `useOriginalToken` (optional): if value is `true` it will send the token of the original session for retry cancels and transaction calls (instead of the latest which is sent by default)

SoftSwiss Adapter (deprecated in favour of SoftSwiss v2)

adapter: `softswiss` config:

```
{
  "secretKey": "abc",
  "url": "http:/example.com"
}
```

- `secretKey` : secret key for wallet authentication
- `url` : wallet endpoint
- `includeProviderInGame` (optional): should be `true` for legacy game naming convention `{provider}:{game}`
- `timeout` (optional): request timeout in seconds (default `15` seconds)

Settings: - `retriesExpiryHours` set to `72` hours

SoftSwiss v2 Adapter

id: (they don't want to use softswiss in the URL, so we use "a8r" which is their internal codename) adapter: `softswiss2` config:

```
{
  "brands": {
    "my-brand1": {"url": "http:/example.com/brand1", token: "abc"},
    "my-brand2": {"url": "http:/example.com/brand2", token: "def"}
  }
  "convertCurrencies": {
    "btc": "ubtc",
    "eth": "meth",
    "bnb": "mbnb",
    "ltc": "mltc"
  }
}
```

- `brands`: object containing brands (`casino_id`'s) and corresponding endpoint urls and auth tokens
- `includeProviderInGame` (optional): should be `true` for legacy game naming convention `{provider}:{game}`
- `timeout` (optional): request timeout in seconds (default `15` seconds)
- `convertCurrencies` (optional): given SoftSwiss doesn't support `ubtc`, but only `btc` (which is not very readable and not all maths support 2+ decimals) we suggest to covert those players in the wallet. So in our platform we will see player with `ubtc` and softswiss will see `btc`.

OpenBox Adapter

adapter: `openbox` config:

```
{
  "secretKey": "abc",
  "vendorUid": "xyz",
  "url": "http:/example.com"
}
```

- `secretKey`: secret key for wallet authentication
- `vendorUid`: vendorUid provided by OpenBox
- `url`: wallet endpoint
- `timeout` (optional): request timeout in seconds (default `10` seconds)

Settings:

- `autoCompleteDisabled`: `true`
- `retriesExpiryHours`: `24`

iSoftBet Adapter

Game id's on the wallet's side should be in the `{provider}.{game}` format (i.e. `my-provider.my-game`)

id: `isoftbet` config:

```
{
  "secretKey": "abc",
  "url": "https://abc.isoftbet.com/rest/service/111",
```

```
  "sessionExpiryMinutes": 2
}
```

- `secretKey`: secret key for wallet authentication
- `url`: wallet endpoint
- `sessionExpiryMinutes` (optional): time in minutes (default: `5`) after which the session is ended in case of inactivity

The integration doesn't support jackpots

Settings:

- `autoCompleteDisabled`: `true`
- `retriesExpiryHours`: `24`

Playtech POP Adapter

Important: since this wallet doesn't use secret keys for identification, you must provide whitelisted IPs

adapter: `playtech`

config:

```
{
  "url":"https://playtech.com",
  "gsId":"123",
  "mfgCode":"ABC",
  "key": "-----BEGIN RSA PRIVATE KEY----- ...",
  "cert": "-----BEGIN CERTIFICATE-----  ...",
  "passphrase": "123",
  "gameVariants": {
      "myGame": [
        {
            "paytableId": "default",
            "paytableTitle": "My variant",
            "paytableDesc": "My variant description",
            "minPaybackPct": 96.1,
            "maxPaybackPct": 96.5
        },
        ...
      ],
      ...
  }
}
```

- `url`: wallet endpoint
- `mfgCode`: id provided by Playtech
- `gsId`: id provided by Playtech
- `timeout` (optional): request timeout in seconds (default `30` seconds)
- `key`: TLS key converted to a single line (typically from `x.key.pem` file)
- `cert`: TLS cert converted to a single line (typically from `x.cert.pem` file)
- `passphrase`: TLS passphrase
- `gameVariants`: object indexed with `game` and with an array of available variants

Variants object:

Structure follows Playtech Marketplace paytable object

- `paytableId`: variant id (`default` for empty variant)

- `paytableTitle`: title of the variant
- `paytableDesc`: description of the variant
- `minPaybackPct`: min RTP %
- `maxPaybackPct`: max RTP %
- `volatilityIndex` (optional): volatility measure
- `confidenceInterval` (optional): confidence interval

Settings:

- `hideReplayBalance`: `true`
- `autoCompleteDisabled`: `true`
- `retriesExpiryHours`: `24`

Relax adapter

adapter: `relax`

config:

```
{
  "url": "https://dev-p2p-cdn.api.relaxg.net/p2p/v2",
  "user": "basic-authentication-user",
  "password": "basic-authentication-password",
  "platformCode": "assigned-platform-code",
  "providers": {
    "test-provider-0": {
      "code": "tp0",
      "name": "Test Provider Zero"
    },
    "test-provider-1": {
      "code": "tp1",
      "name": "Test Provider One"
    },
  "currencyAliasesPerBrand": {
    "GC.": {
      "10": "gc-1000000",
      "20": "gc-1000"
    }
  },
}
```

- `url`: wallet endpoint
- `user`: Basic access authentication User, used for both ways communication
- `password`: Basic access authentication Password, used for both ways communication
- `platformCode`: platform id assigned for the environment by Relax
- `providers`: In order to make provider's games enabled for Relax, an entry in `providers` map needs to be created, with `code` assigne by the Relax, and human-readable `name` for a given provider.
- `currencyAliasesPerBrand`: used to map Relax currencies to different fixed rate multipliers currencies per brand

Security:

Relax communication requires "Basic access authentication" (`user` / `password` config entries are used symmetrically for operator/platform requests). It's required to whitelist Relax endpoint in wallet configuration, as well as provide Relax with platform IP for whitelisting.

Auto Complete disabling:

Relax authentication ticket expires very quickly making it impossible for auto complete system to finalise the transactions. Autocompletion needs to be disable (`autoCompleteDisabled` : `true`) and rounds resolved manually (e.g. from their side via `finalize` endpoint).

FEIM:

Relax requires platforms to comply with the Front End Integration Module standards to enable communication through the operator's `iframe` . To ensure this feature operates correctly, the client application must use the following methods of the Connector: `setActiveBet` , `gameLoaded` , `muted` , `unmuted` , `exit` .

Settings:

- `autoCompleteDisabled` : `true`
- `depositRetries` : `1,5,15,30,1`
- `cancelRetries` : `1,5,15,30,1`
- `hideReplayBalance` : `true`

And for Pokerstars:

- `autoCompleteDisabled` : `true`
- `depositRetries` : `0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.85,0.85,0.85,0.85,0.85,0.85,0.85,0.85,0.85,0.85,6`
- `cancelRetries` : `0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,0.1,6`
- `retriesExpiryHours` : `24`

Currencies mapping into different multiplier aliases:

In order to map currency (e.g. `GC.`) for a given brand (e.g. `10`) it is required to:

- add mapping `relax currency id -> brand -> platform currency alias` to the `currencyAliasesPerBrand` (like in the example),
- add `gc-1000000` as Alias (of e.g. `eur` with multiplier `0` for fun currencies),
- add Fixed Rate for this specified currency `gc-1000000` with the desired multiplier (e.g. `1000000`) and the original currency symbol (`GC.`)

Light & Wonder adapter

adapter: `lnw`

config:

```
{
  "url": "https://ogs-gpapi-aws-3pp-int-0.nyxaws.net/gpapi",
  "username": "basic-authentication-user",
  "password": "basic-authentication-password",
  "gpid": "1234",
  "authUrl": "https://auth-service.dev.auth-service.fft.nyxop.net",
  "creditApiUrl": "https://ogs-papi-aws-3pp-int-0.nyxaws.net",
  "creditApiClientId": "5lm4r24i69uc03chqnj1qquvtd",
  "creditApiClientSecret": "1hb59om5i6fbecvio2jpre4n1j2htta4i77oatjn6mlbarmud2hn"
  "ogsGameIdsMapping": {
    "123456": "myGame"
  }
}
```

- `url` : wallet endpoint
- `username` : Basic access authentication User, used for both ways communication
- `password` : Basic access authentication Password, used for both ways communication

- `gpid` : Game provider id assigned by Light & Wonder
- `authUrl` : Authentication url used for Credit API assigned by Light & Wonder
- `creditApiUrl` : Credit API url assigned by Light & Wonder
- `creditApiClientId` : Credit API client id assigned by Light & Wonder
- `creditApiClientSecret` : Credit API client secret assigned by Light & Wonder
- `ogsGameIdsMapping` : object indexed map from `ogsgameid` to a `game`

GCM:

Light & Wonder requires platforms to comply with the GCM standards to enable communication through the operator's `iframe` . To ensure this feature operators correctly, the client application must use the following methods of the Connector: `gameLoaded` , `mute` , `unmuted` , `exit` , `turboToggle` (optional), `paytableToggle` (optional), `helpToggle` (optional), `aboutToggle` (optional), `startAutoplay` , `stopAutoplay` , `freeze` , `unfreeze` , `freezeBet` , `unfreezeBet` .

BetConstruct Adapter

adapter: `betconstruct` config:

```
{
  "secretKey": "abc",
  "url": "http:/example.com"
}
```

- `secretKey` : secret key for wallet authentication
- `url` : wallet endpoint
- `includeProviderInGame` (optional): should be `true` for legacy game naming convention `{provider}:{game}`
- `timeout` (optional): request timeout in seconds (default `3` seconds - advised by them)

Fizzy Bubbly (N2) Adapter

adapter: `fizzybubbly` config:

```
{
  "secretKey": "abc",
  "publicKey": "def",
  "url": "http:/example.com"
}
```

- `secretKey` : secret key for wallet authentication
- `publicKey` : public key for wallet authentication
- `url` : wallet endpoint
- `includeProviderInGame` (optional): should be `true` for legacy game naming convention `{provider}:{game}`
- `timeout` (optional): request timeout in seconds (default `15` seconds)

QTech Adapter

adapter: `qtech` config:

```
{
  "secretKey": "abc",
  "gameLaunchPassKey": "def",
  "gameResultPassKey": "ghi",
```

```
  "url": "http:/example.com"
}
```

- `secretKey`: secret key for wallet authentication
- `gameLaunchPassKey`: public key for game launch
- `gameResultPassKey`: public key for game replay
- `url`: wallet endpoint
- `includeProviderInGame` (optional): should be `true` for legacy game naming convention `{provider}:{game}`
- `timeout` (optional): request timeout in seconds (default `15` seconds)

Reevo Adapter

adapter: `reevo` config:

```
{
  "secretKey": "abc",
  "callerID": "def",
  "callerPass": "ghi",
  "salt": "123",
  "url": "http:/example.com"
}
```

- `callerID`: caller ID (api_login)
- `callerPass`: caller password (api_password)
- `salt`: salt value
- `url`: wallet endpoint
- `includeProviderInGame` (optional): should be `true` for legacy game naming convention `{provider}:{game}`
- `timeout` (optional): request timeout in seconds (default `15` seconds)

Slotegrator Adapter

adapter: `slotegrator` config:

```
{
  "secretKey": "abc",
  "url": "http:/example.com"
}
```

- `secretKey`: secret key for wallet authentication
- `url`: wallet endpoint
- `includeProviderInGame` (optional): should be `true` for legacy game naming convention `{provider}:{game}`
- `timeout` (optional): request timeout in seconds (default `15` seconds)

Grrr Adapter

adapter: `grrr` config:

```
{
    "brands": {
      "brand1": {
          "url": "https://example1.cpm",
          "secretKey": "testKey"
      },
```

```
      "brand2": {
          "url": "https://example12.cpm",
          "secretKey": "testKey2"
      }
    },
    providerPartnerId: "shadylady",
    thumbnailUrl: "https://dupa.com/${game}/thumbnail.png",
};
```

- `brands` : object containing brands (`partnerId` 's) and corresponding endpoint urls and secret keys
- `providerPartnerId` : identifier of partner
- `thumbnailUrl` - url for retrieving game image, url must contain `${game}` which will be replaced by game id
- `timeout` (optional): request timeout in seconds (default `15` seconds)

PinUp Adapter

adapter: `pinup`

config:

```
{
    url: "https://dev.pinup.net/walletapi",
    providerId: "wickedgames",
    providerToken: "providerTestToken",
    secretKey: "testSecretKey",
};
```

- `url` : pinup wallet url
- `providerId` : identifier of games provider
- `providerToken` : a token that is used for authorization to pinup wallet
- `secretKey` : a key that is used in authorization header for requests to the wallet
- `timeout` (optional): request timeout in seconds (default `15` seconds)

Slotify Adapter

This integration allows to plug into another Slotify instance.

adapter: `slotify` config:

```
{
  "secretKey": "abc",
  "url": "http:/example.com/rgs/your-rgs"
}
```

- `secretKey` : secret key for wallet authentication
- `url` : wallet endpoint
- `timeout` (optional): request timeout in seconds (default `15` seconds)

On the Slotify instance which will be playing a role of a wallet you need to configure RGS with `standard` adapter and for urls (`funUrl` , `realUrl`) use: `http://example.com/launch/fun?operator=${wallet}:@${key}` to pass end wallet and end key.

BadHombre Adapter

id: `badhombre` config:

```
{
    "privateKey": "-----BEGIN PRIVATE KEY-----\n ... \n-----END PRIVATE KEY-----",
    "brands": {
      "brand1": {
          "url": "https://example1.cpm",
          "publicKey": "-----BEGIN PUBLIC KEY-----\n ... \n-----BEGIN PUBLIC KEY-----"
      },
      "brand2": {
          "url": "https://example12.cpm",
          "publicKey": "-----BEGIN PUBLIC KEY-----\n ... \n-----BEGIN PUBLIC KEY-----"
      },
    }
```

- `privateKey` : private key (pkcs8)
- `brands` : set of brands with public keys and urls
- `urls` set of urls per brand

Settings:

- `retriesExpiryHours` : `24`

Custom wallets adapters

There is a possibility to create custom Wallet adapters, but of course it requires writing code with custom implementation.

RGS's

RGS's are remote endpoints to play games (server-to-server integrations).

Standard RGS

Provider integration via standard API is documented here.

Custom RGS integrations

There is a possibility to create custom RGS integration, but of course it requires writing code with custom implementation.

Currencies

The platform supports multiple currencies. Base currency is set to `eur`.

Feeds

Currency exchange rates are downloaded every day (rates from the day before).

Two feeds are used:

- `currencylayer.com` for FIAT currencies (free and paid plans available, requires an API key)
- `coinapi.io` for cryptocurrencies (free and paid plans available, requires an API key)

Aliases

There is possibility to create aliases with fixed rate to another currency taken from the feed. The common use-cases would be:

- stable coins like `usdt` which is `1:1` conversion to `usd`
- currencies with large decimals like `btc` being `1000000:1` to `ubtc`
- currencies with large amounts like `kvnd` being `1:1000` to `vnd`
- virtual currencies like i.e. chips or bananas

Reporting

Aggregation Process

Every hour there is a transaction aggregation process triggered which aggregates hourly data into a single row for improved query speed. That aggregated report is called Game Win (query `gameWin`).

Occasionally, it can happen that a withdrawal is canceled in another hour and therefore the aggregate hourly aggregate will incorrectly include canceled withdrawal. Therefore, it is advised to clear all the data from given month before sending invoices to the clients. There is GraphQL mutation (`regenerateGameWin`) that regenerates the game winnings data since selected time.

Expensive query optimisations

There are two main types of queries that can be very slow and affect the system. In both cases system will analyse the query cost and decide on the fallback method:

- if sorting large data set would take too expensive it will return an error
- if getting total number of rows needed for pagination would be too expensive it will switch to prev/next pagination without ability to jump to any page

GraphQL API stitching

The adapter service can stitch external GraphQL API. Two main benefits of that are:

- single endpoint to cover APIs from multiple services
- centralised authentication and permission management
- centralised audit logs

To configure new API to stitch you need to add the URL to it in `GRAPHQL_ENDPOINTS` environment variable. You can pass multiple URLs separated with comma (`,`).

Also, because GraphQL introspection doesn't expose custom directives you need to expose `schema` query which will return GraphQL schema string.

```
type Query {
    schema: String!
}
```

```
{
    Query: {
        schema: () => {
            return fs.readFileSync(process.cwd() + "/graphql/schema.graphql").toString();
        }
    }
}
```

Account is passed via context

```
type IContext = {
    account: {
        email?: string;
        permissions?: string[];
        rgss?: string[];
        providers?: string[];
        wallets?: string[];
        operators?: string[];
        brands?: string[];
    }
}

const resolvers = {
    Query: {
        async test(_, _, {account}: IContext) {
            console.info(account.email);
            return true;
        },
    }
};
```

Accounts

Account and passwords lifecycle

After creating new account the email is sent to the specified account containing a link to set up a password.

If account password is lost it can request an email with a link to change the password.

Permissions

Each account contain set of independent permissions assigned to the account. GraphQL API queries and mutations check if particular account has specified permission. In below example query `test` will be available only to accounts which contain `testPermission`. For those who don't have such permission, the method will not be visible in API schema, and it will not be possible to call it.

```
directive @auth(permission:String!) on FIELD_DEFINITION
type Query {
    test: Boolean! @auth(permission: "testPermission")
}
```

Audit logs

Each query and mutation call can be logged for audit purposes using `@log` directive. Parameters `variables` and `results` indicate if those should be saved too or skipped.

```
directive @log(variables:Boolean result:Boolean ) on FIELD_DEFINITION
type Query {
    test: Boolean! @log(variables: true result: true)
}
```

Filters (rgs, provider, wallet, operator, brand)

Each account can specify multiple "tags" under rgs, provider, wallet, operator, brand. Those "tags" are used to data filtering. Imagine an account with the following configuration:

email	wallets	providers	operators	brands	rgss
test@example.com	`null`	`["myProvider"]`	`["casinoA", "casinoB"]`	`null`	`null`

Given account will have access to the data (i.e. transactions) that fulfill given condition

```
(provider === "myProvider") && (operator === "casinoA" || operator === "casinoB")
```

Wallet Verifier

Wallet Verifier is a tool that automatically simulates popular user scenarios by sending requests to the remote wallet and validating responses.

To run it you need to enter wallet name, operator name and key. Key is generated by the remote wallet. It can be obtained by requesting it from the wallet owner or by extracting it from the url the game is embedded with. Either way there is no automated way of getting it.

Round Verification

Transaction verification happens in real time and consist of three steps

1. API validation

Each Transaction is validated against below criteria. If any of these conditions are positive, the transaction is rejected and email sent marked with high priority. These conditions should NEVER be positive, so if it happens, it means a bug in the code or integration, and it should be immediately investigated to understand the root cause.

- All transactions from given `roundId` belongs to the same player
- There is no transaction associated with given `roundId` after the round is finished (`roundFinished: true`)
- All transactions from given `roundId` belongs to the same game (only for transaction type `normal`)
- Number of withdrawals per round doesn't exceed `MAX_WITHDRAWS` (default: 1)
- Number of deposits per round doesn't exceed `MAX_DEPOSITS` (default: 1)
- Deposit transaction has corresponding withdrawal for given `roundId` (only for a transaction type different from `promo`)
- The withdrawal amount is bigger than zero
- The deposit amount is bigger or equal to zero

2. Alerting

Email notification when meeting certain criteria (only if `ALERT_ENABLED` is set):

- Win amount equal or greater than `ALERT_MAX_WIN`
- Win ratio to base bet equal or greater than `ALERT_MAX_WIN_RATIO`

3. Score verification

For each win, bigger than a bet, we calculate the score which is used to detect suspicious patterns.

The score is calculated as a sum of points. Only one check per category is processed.

If the score is bigger than **75**, notification email is sent. If the score is bigger than **100**, the transaction is rejected and user presented with the following error message: `Your win requires manual verification. Please contact Support team.`

After manual verification the transaction can be released using `Mark as failed` button in the Back Office which will make it subject to be auto closed during `1h` cron task.

If there are **2** rejected transactions from the same player, we block the player and send an email.

If there are **3** rejected transactions from the same wallet, we block the wallet and send an email.

Once the transaction is rejected, the team should run manual verification to check if the transaction was legit:

- if the transaction was legit, you can click "Mark as failed" in the back office which will give player time to complete it manually or it will be auto completed as any other failed transaction
- if the transaction was not legit, most likely you should change it's status to `voided`. This is very rare transaction and it can be done only from the sql query

Configuration

Configuration can be applied to each RGS, Wallet and Game separatly and then inspection config is merged from all, with the following order (higher number overwrites lover, so the Wallet config is "the most important"):

1. RGS inspection config
2. Game inspection config
3. Wallet inspection config

Config consists of three sections refering to Round verification steps.

```
{
  validation: {
    maxWithdraws: 1,
    maxDeposits: 1,
  },
  alerting: {
    maxWin: 10000,
    maxWinRatio: 5000,
  },
  verification: {
    alertedThreshold: 75,
    rejectedThreshold: 100,
    maxPlayerRejectedTransactions: 2,
    maxWalletRejectedTransactions: 3,
    rules: {
      playerRegistration: [
        {operator: "<", value: 2, points: 15}, //player registered in less then 2 hours will get 15 points
        {operator: ">", value: 24, points: -5}, //player registered before 24 hours ago will get -5 points
      ],
      //more rules here
    },
  },
};
```

Rules

Each rule can have multiple conditions which are processed synchronously like `if..else if` conditions. Once a condiion is fullfilled the system moves to the next rule.

Rule	Description
`playerRegistration`	Hours since player's first registration
`transactionOffset`	Time offset in milisseconds since previous transaction in the round

Rule	Description
`consecutiveWinsWithSameAmount`	Number of consecutive wins with the same amount
`winFrequencyShortTerm`	Win frequency over last 25 rounds
`playerRTPLongTerm`	Player's RTP over last 90 days
`playerGameWinMediumTerm`	Player's Game Win over last 7 days excluding top 3 wins
`playerShortTermNormalizedRTP`	Player's normalized RTP over last 100 rounds excluding top 2 wins
`winAmount`	Round's win amount
`numberOfLargetNetWins`	Number of net wins (win-bet) over €10k in last 2 days
`maxWinRatio`	Ratio to thoeretical max win (only for internal RGS)
`walletGameWinShortTerm`	Wallet's Game Win in last 24 hours

If there is no sufficient history is available, the check may be skipped.

Report Sender

Report sender is a functionality to configure a periodic report send-out. Currently, reports can be delivered to defined email with attachment. In the future we may add more transport like SFTP or Webhook.

Report	Description	Variables	Suggested Cron
`DGE_reports`	DGE report for New Jersey market	{wallet: "my-wallet"}	`0 12 * * *` every day at 12:00 CET